

Faculty of Engineering

Internal Design Document

# MQTT Protocol Design for Distributed Real-Time Telemetry System for QUT Microgrid


**Date:** October 12, 2026

**Student Engineer:** Dylan Hessing

**Student ID:** n11245310

**Supervisor:** Assoc. Prof. Geoff Walker


**Version History**

| Version | Date | Author | Changes / Comments |
|---|---|---|---|
| 1 | April 20, 2026 | Dylan Hessing | Initial Design |
| 2 | October 11, 2026 | Dylan Hessing | Updated to as-built implementation |

# Contents

**1 Overview** **2**

**2 Scope** **2**

**3 Broker Selection & Configuration** **2**

**4 Topic Hierarchy** **3**

4.1 Naming Scheme . . . . . 3

4.2 Topics Published . . . . . 4

4.3 Wildcard Subscription Patterns . . . . . 4

**5 Payload Format** **4**

5.1 Format Selection . . . . . 4

5.2 Burst Payload . . . . . 5

5.3 PTP Status Payload . . . . . 6

5.4 Full-Rate Binary Payload . . . . . 6

**6 Quality of Service Levels & Retained Messages** **7**

6.1 Quality of Service Strategy . . . . . 7

6.2 Retained Messages . . . . . 7

6.3 Node Liveness . . . . . 8

**7 Client Architecture** **8**

7.1 DAQ Nodes (STM32 Publishers) . . . . . 8

7.2 Historian Subscribers . . . . . 9

**8 Security** **10**

# 1 Overview

This document describes the MQTT protocol design for the distributed real-time telemetry system developed as part of the EGH490 thesis project at the Queensland University of Technology. The system acquires electrical measurements from STM32-based data acquisition nodes deployed across the QUT microgrid and transports them to a centralised time-series historian via an MQTT message broker.

The design covers broker selection and configuration, topic hierarchy, payload format, quality of service strategy, client architecture, security posture, and historian integration. All decisions are scoped to a two-node validation testbed operating over a wired Ethernet LAN, with the architecture designed to generalise to an arbitrary number of nodes without structural change.

Version 2 of this document records the system as built. Where the implementation departed from the original design, the as-built behaviour is described, and design elements that were not implemented are listed as deferred in the relevant section.

# 2 Scope

The following constraints apply to all documented design decisions.

Table 1: Design Constraints

| Parameter | Value |
|---|---|
| Transport medium | Wired Ethernet (LAN) only |
| Number of DAQ nodes (validation testbed) | 2 (`node_01`, `node_02`) |
| DAQ node MCU | STM32C5A3ZGT6 (IEEE 1588 hardware timestamping) |
| Node software | FreeRTOS, lwIP TCP/IP stack and lwIP MQTT client |
| Time synchronisation protocol | PTP (IEEE 1588), peer-to-peer delay mechanism |
| Historian | InfluxDB v2 |
| Target measurement standards | IEC 61000-4-30 Class S, IEEE 1459-2010 |

Wireless transport is specifically out of scope. The wired-only constraint eliminates RF-induced jitter as a variable in timestamp accuracy, permitting PTP synchronisation to operate within the sub-microsecond range achievable on a switched Ethernet topology. The two-node testbed reflects hardware availability at the time of validation; the architecture imposes no upper bound on the number of nodes.

# 3 Broker Selection & Configuration

Eclipse Mosquitto 2 is selected as the MQTT broker. This is suitable for a research testbed with two DAQ nodes as it is:

- lightweight
- well documented

- suitable for co-deployment on the historian server alongside InfluxDB without significant resource contention
- carries no licensing constraints under the Eclipse Public Licence

Alternatives such as EMQX and HiveMQ were considered but are designed for large-scale deployments with thousands of concurrent clients; the additional operational complexity is not justified at this scale.

The broker runs as the `eclipse-mosquitto:2` Docker container on the historian server (see the InfluxDB design document for the full Docker Compose deployment). The as-built configuration (`mosquitto/config/mosquitto.conf`) is:

```
listener 1883
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

Port 1883 is used for lab LAN traffic. Network exposure is controlled by the Docker port mapping rather than the listener address: the default mapping `127.0.0.1:1883:1883` admits only clients on the server itself (such as a test message published with `mosquitto_pub`), and must be changed to `1883:1883` before DAQ nodes can connect. Anonymous connections are currently permitted; see Section 8. Persistence is enabled, although with all telemetry published at QoS 0 there are no queued messages for it to preserve.

**Deferred** The originally designed authentication (`password_file`), explicit LAN interface binding, and `max_inflight_messages`/`max_queued_messages` limits were not applied. TLS on port 8883 remains a future hardening measure.

# 4 Topic Hierarchy

## 4.1 Naming Scheme

Topics follow a four-level hierarchy:

```
microgrid/<node_id>/<subsystem>/<metric>
```

Table 2: Hierarchy Description

| Segment | Description | Values |
|---|---|---|
| `node_id` | Fixed, zero-padded node identifier | `node_01`, `node_02` |
| `subsystem` | Quantity category | `voltage`, `current`, `ptp_status` |
| `metric` | Specific measured quantity | `phase_a`, `phase_b`, `phase_c` |

Node identifiers are fixed strings assigned at firmware flash time (`NODE_ID` in `app/node_config.h`). MAC addresses and IP addresses are explicitly avoided as node identifiers as both are mutable and would inval-

idate historical records if hardware is replaced. Additional subsystems (for example `power` or `frequency`) can be introduced without structural change if the firmware is extended to publish derived quantities.

### 4.2 Topics Published

Each node publishes six telemetry topics:

```
microgrid/node_01/voltage/phase_a
microgrid/node_01/voltage/phase_b
microgrid/node_01/voltage/phase_c
microgrid/node_01/current/phase_a
microgrid/node_01/current/phase_b
microgrid/node_01/current/phase_c
```

and, on every PTP Sync, one status topic (Section 5.3):

```
microgrid/node_01/ptp_status/servo
```

and, once per connection, the firmware version, retained (Section 6.2):

```
microgrid/node_01/status/firmware
```

Optionally, when built with `MQTT_PUBLISH_DAQ2`, a node also publishes its full-rate binary stream on `daq2/<node_id>/samples` (Section 5.4). This topic is deliberately outside `microgrid/#`, so that the historian bridge, which expects JSON, does not subscribe to it.

### 4.3 Wildcard Subscription Patterns

Table 3: Wildcard Subscription Patterns

| Pattern | Use Case |
|---|---|
| `microgrid/#` | Full subscription: used by the historian bridge |
| `microgrid/node_01/#` | All measurements from a single node (debugging) |
| `microgrid/+/voltage/#` | All voltage measurements across all nodes |
| `daq2/+/samples` | Full-rate binary streams from all nodes |

## 5 Payload Format

### 5.1 Format Selection

Telemetry payloads are serialised as UTF-8 JSON. This prioritises developer ergonomics and debuggability: payloads are human-readable, trivially inspected with any MQTT client, and require no schema negotiation between publisher and subscriber. A compact binary format is used only for the optional full-rate stream (Section 5.4).

### 5.2 Burst Payload

The ADC sample rate was fixed at approximately 30 kHz per channel, which resolves the original choice between single-sample and burst publishing in favour of bursts. Every 5 s, each node publishes one 50 Hz cycle of each channel, decimated to approximately 1 kHz: 20 samples per message, one message per channel.

```
{
  "ts": 1791345068887514368,
  "sync": 1,
  "dt_ns": 998424,
  "v": [231.45, 229.80, 224.12, ...]
}
```

Table 4: Burst Payload Description

| Field | Type | Description |
|---|---|---|
| `ts` | integer | Unix (UTC) time in nanoseconds of the first sample, from the node's PTP-disciplined clock |
| `sync` | integer | 1 if the node's PTP servo was locked when the burst was captured, otherwise 0 |
| `dt_ns` | integer | Interval between samples in nanoseconds |
| `v` | number[20] | Sample values in SI units (V or A), to two decimal places |

Sample $k$ was taken at $t_s + k \cdot \Delta t$, where $t_s$ is `ts` and $\Delta t$ is `dt_ns`. The interval is the nearest whole number of ADC frames to 1 ms (for example 998 424 ns at 30 048 frames/s), so it is reported explicitly rather than assumed.

Implementation notes:

- The PTP clock runs on TAI. `ts` is converted to UTC by subtracting the TAI−UTC offset (`currentUtcOffset`) from the grandmaster's Announce messages, or 37 s until one has arrived. No offset is subtracted if the grandmaster announces an arbitrary (ARB) timescale.
- `ts` is the centre of the first sample's accumulated ADC burst, derived from the hardware timer and PTP clock rather than from interrupt timing.
- Currents are sampled one ADC slot after the voltages, so each current message's `ts` is offset from the voltage messages by that interval.
- `sync` is sent as an integer rather than a JSON boolean, because the historian bridge's JSON parser drops booleans.
- A burst is started only on data whose timestamp is valid, and is restarted if a frame gap or PTP clock step occurs during collection, so the 20 samples are always contiguous and on a single time base.

Units are implied by the topic rather than included in the payload: a message on `microgrid/node_01/voltage/phase_a` carries volts.

The single-sample payload of the original design (`{"ts", "v"}`) is not used.

### 5.3 PTP Status Payload

On every PTP Sync, about once per second, each node publishes the result of its servo update on `microgrid/<node_id>/ptp_status/servo`:

```
{
  "ts": 1791696297692880523,
  "offset_ns": 18274,
  "path_delay_ns": 15497,
  "freq_ppb": 18010,
  "state": 2,
  "locked": 0,
  "stepped": 0,
  "utc_offset_s": 37,
  "utc_announced": 1
}
```

Table 5: PTP Status Payload Description

| Field | Type | Description |
|---|---|---|
| `ts` | integer | The grandmaster's time for that Sync, as Unix (UTC) nanoseconds, using the same TAI offset as the bursts |
| `offset_ns` | integer | Measured clock offset from the grandmaster |
| `path_delay_ns` | integer | Mean path delay used in the offset calculation |
| `freq_ppb` | integer | Frequency correction applied by the servo, relative to the nominal clock rate |
| `state` | integer | Servo state: 0 no Sync yet, 1 estimating frequency, 2 PI loop running |
| `locked` | integer | 1 once eight consecutive offsets are within 1 µs |
| `stepped` | integer | 1 if this update stepped the clock (offset above 1 ms) |
| `utc_offset_s` | integer | TAI−UTC offset subtracted from `ts` and the bursts' timestamps |
| `utc_announced` | integer | 1 if that offset came from the grandmaster's Announce, 0 if it is the 37 s default |

The values are 64-bit integers formatted by the firmware itself, since its `printf` lacks 64-bit support; offsets before the first clock step can exceed $10^{18}$ ns. As for the bursts, booleans are sent as integers.

### 5.4 Full-Rate Binary Payload

Unless built with `MQTT_PUBLISH_DAQ2` off (it is on by default), each node additionally publishes every acquisition packet on `daq2/<node_id>/samples`. The payload is a 128-byte little-endian header followed by *frames* × *channels* `float32` values in row order. The header carries a sequence number and frame index (so gaps reveal dropped data), the timestamp of the first frame and the frame period, per-channel

sampling offsets, the PTP epoch (which changes on every clock step), a timing uncertainty estimate, and flags indicating PTP lock and timestamp validity. The flags also carry the grandmaster's timescale and, in their upper 16 bits, the TAI−UTC offset the node received in its Announce messages. The full layout is defined by `daq_packet_t` in the firmware's `app/daq/daq_task.h`. Publishing is best-effort: a packet that cannot be queued is dropped, as sampling never waits on the network. The stream is ingested by the `daq2-ingest` service, described in the InfluxDB design document.

# 6 Quality of Service Levels & Retained Messages

## 6.1 Quality of Service Strategy

MQTT defines three quality of service (QoS) levels governing delivery guarantees between publisher and broker, and between broker and subscriber. All messages published by the nodes use QoS 0.

Table 6: QoS Strategy

| Message Type | QoS | Rationale |
|---|---|---|
| Telemetry bursts | 0 (fire and forget) | ACK overhead is disproportionate to benefit; a missed burst is bounded and tolerable; retransmitting a stale sample is actively harmful to the historian. |
| PTP status | 0 (fire and forget) | One message per Sync, about once a second; the next message supersedes a lost one. |
| Full-rate binary packets | 0 | High rate; the acquisition pipeline must never block on the network. |

QoS 2 (exactly once) is not used as the four-way handshake overhead is rarely justified in practice.

## 6.2 Retained Messages

Measurements are not retained, as retaining high-frequency samples would deliver a burst of stale data to every new subscriber. The one retained message is the firmware version, published once per connection on `microgrid/<node_id>/status/firmware` as `{"ts": <Unix UTC ns>, "firmware": "<git describe>"}`, so that any subscriber can see which firmware each node runs. The version is taken from `git describe` at build time and ends in `-dirty` if the source had uncommitted changes. It is sent after the node's first PTP clock step, so that `ts` is real time, and Telegraf keeps `firmware` as a string field (`json_string_fields`).

On each connection, the firmware publishes an empty retained message to `microgrid/<node_id>/status/heartbeat`. This deletes any retained heartbeat left on the broker by earlier firmware versions, and can be removed once no broker holds one.

### 6.3 Node Liveness

Node liveness is determined on the historian side: the Grafana dashboard reports each node's status from the time since its most recent sample. As every node publishes a burst per channel every 5 s, a node that has not reported within a few intervals is treated as offline. This requires no additional MQTT traffic or firmware logic, and also detects failures upstream of the broker connection, such as a stalled acquisition pipeline, that a broker-mediated mechanism would miss.

**Deferred** The original design used a retained heartbeat topic and a Last Will and Testament (LWT) message for broker-mediated liveness detection, together with QoS 1 events, alarms, and configuration updates. These were replaced by the historian-side approach above, and no event, alarm, or configuration topics exist.

## 7 Client Architecture

### 7.1 DAQ Nodes (STM32 Publishers)

Each STM32 DAQ node operates as an MQTT publisher using the lwIP MQTT client (`lwip/apps/mqtt.h`) on top of the lwIP TCP/IP stack and FreeRTOS. The lwIP client was chosen over Paho Embedded C and FreeRTOS coreMQTT as it is integrated with the network stack already in use, adding no further dependency.

**Publish Path**
Acquisition and publishing run in separate FreeRTOS tasks:

1. The ADCs sample continuously at approximately 30 kHz; DMA fills a ring buffer and signals the DAQ task when a block completes.
2. The DAQ task scales and timestamps each block into a packet and places it on a ready queue. If no packet slot is free, the block is dropped rather than stalling acquisition.
3. The MQTT task is the sole consumer of the ready queue. Every 5 s, it collects one 50 Hz cycle (20 samples at approximately 1 kHz) of each channel across consecutive packets, serialises one JSON burst per channel, and publishes the six messages at QoS 0.
4. Unless built with `MQTT_PUBLISH_DAQ2` off, the MQTT task also publishes every packet in binary on `daq2/<node_id>/samples` (approximately 470 per second), at QoS 0. A packet that cannot be queued in the lwIP output buffer is dropped and counted rather than waited for.
5. After each PTP servo update (on every Sync, about once a second), the MQTT task publishes the servo's state on `microgrid/<node_id>/ptp_status/servo`.
6. Once per connection, after the first PTP clock step, it publishes the firmware version on `microgrid/<node_id>/status/firmware`, retained.
7. Every packet is released back to the pool immediately after use, so acquisition never runs out of slots.

**Connection Management**
Each node uses a static IP address, and the broker address, set in `app/node_config.h`. The MQTT task

attempts a connection at most once per second while the Ethernet link is up, abandons an attempt that has not completed within 10 s, and reconnects automatically after broker loss. If the link goes down, the session is dropped. The keep-alive interval is 60 s. Telemetry is not buffered during disconnection; the resulting gap is visible in the historian, which is acceptable for a research testbed.

Client IDs follow the convention `mqttc-<node_id>`. No credentials are configured, matching the broker's current anonymous access (Section 8).

**Diagnostics**

Once per second, each node prints acquisition, MQTT, and Ethernet counters (connections, published and dropped messages, overruns, queue drops, invalid timestamps) and the PTP lock state to its debug UART.

## 7.2 Historian Subscribers

Two subscribers on the historian server divide the topic space between them: Telegraf subscribes to `microgrid/#` (the JSON bursts and PTP status), and the `daq2-ingest` service to `daq2/+/samples` (the full-rate binary stream).

**Telegraf**

Telegraf with the MQTT consumer input plugin is used as the bridge for the JSON topics. This provides a configuration-driven pathway from the broker to InfluxDB with no custom subscriber code. A regex processor extracts each topic segment into a tag (`node_id`, `subsystem`, `metric`), and the payload's `ts` is used as the point timestamp. The as-built configuration (`telegraf/telegraf.conf`) is:

```
[agent]
  flush_interval = "1s"
  precision = "1ns"

[[inputs.mqtt_consumer]]
  servers = ["tcp://mosquitto:1883"]
  topics = ["microgrid/#"]
  qos = 0
  data_format = "json"
  json_time_key = "ts"
  json_time_format = "unix_ns"
  topic_tag = "topic"

[[processors.regex]]
  order = 1
  [[processors.regex.tags]]
    key = "topic"
    pattern = "^microgrid/([^/]+)/([^/]+)/([^/]+)$"
    replacement = "${1}"
    result_key = "node_id"
  # ... likewise ${2} -> subsystem, ${3} -> metric

[[outputs.influxdb_v2]]
```

```
urls = ["http://influxdb:8086"]
token = "${INFLUXDB_TOKEN}"
organization = "qut-microgrid"
bucket = "telemetry"
```

Telegraf's JSON parser flattens the `v` array into fields `v_0` to `v_19`, so each burst is stored as a single InfluxDB point at the first sample's timestamp. A Starlark processor additionally splits each burst into 20 points, one per sample at `ts` $+k\cdot$ `dt_ns`. PTP status messages pass through unchanged as one point per Sync. The resulting schema is described in the InfluxDB design document.

**daq2-ingest**

Telegraf cannot compute per-sample times from the binary packet header, so the full-rate stream has a dedicated subscriber: `daq2-ingest`, a Python service using the Eclipse Paho client. One receiver process subscribes to `daq2/+/samples` at QoS 0 and hands each node's packets to a worker process, which decodes them and writes them to InfluxDB. The service is described in the InfluxDB design document.

## 8 Security

The system operates on a university research lab LAN and is not exposed to the public internet; accordingly, the security measures are pragmatic rather than exhaustive. Security was planned in three phases. As built, the system is at an earlier stage than Phase 1.

**Current State**

- The broker accepts anonymous connections; nodes connect without credentials.
- Exposure is limited by the Docker port mapping: by default the broker is reachable only from the historian server itself, and is opened to the lab LAN only when real nodes are connected.

**Phase 1: Authentication (deferred)**

- Disable anonymous connections.
- Enable username and password authentication via Mosquitto's built-in password file mechanism.
- Issue separate credentials per client (one per DAQ node, one for the Telegraf bridge).

**Phase 2: Hardening (deferred)**

Access control lists (ACLs) enforce the principle of least privilege. Each DAQ node may only publish to its own topic subtree, so that a compromised node cannot spoof measurements from another node. The Telegraf bridge may subscribe to the full topic tree but cannot publish.

**Phase 3: TLS (future)**

If the broker is ever exposed outside the lab VLAN, mutual TLS on port 8883 with certificates issued by a local certificate authority is the intended upgrade path. This is not required for the current testbed scope.